MANDISETU

Market Intelligence & Transaction Enablement Platform

Project Report, System Design & Prototype Status

A bridge between farm-gate produce and fair market outcomes

Frontend	React + Vite
Backend	Node.js + Express REST API starter
Database	MySQL schema
Market intelligence	Sample data; live feeds and ML not connected
Prototype status	Interactive browser demo; some features simulated

1. Executive Summary

MandiSetu is a market-intelligence and transaction-enablement platform designed to bridge the information gap between smallholder farmers/FPOs and buyers. It aims to combine market-price context, cost-aware sale decisions, buyer trust, produce lot creation, aggregation, logistics and transparent transaction records.

2. Problem Statement

A headline price does not tell a farmer the amount they will retain after transport, commission, storage and other costs. Farmers and FPOs also need reliable market context, trusted counterparties, larger-volume aggregation, and clearer transaction/dispute records.

3. Objectives

• Aggregate live and historical mandi prices with source, date and market.

• Forecast short-term price trends and recommend possible sale windows with uncertainty.

• Compare net realization after transport, commission, storage and other costs.

• Enable digital lots, quality details, buyer verification and reliability indicators.

• Support FPO-level aggregation, logistics coordination, transaction tracking and dispute resolution.

4. Website Pages

1. Market Overview

Summary of sample lots, buyer profiles, transactions and cost-aware selling concept.

2. Market Intelligence

Sample lot-price board and illustrative chart. No live mandi data or verified historical records.

3. Sell-Time Advisor

Compares a sell-now calculation with a user-entered later-price scenario less holding cost. This is not an ML prediction.

4. Net Realization

Calculates quantity × offer price minus transport, commission, storage and other costs.

5. Digital Lots & Quality

Creates demo lots with crop, variety, grade, quantity and location. No actual quality certification.

6. Buyer Network

Displays sample buyer cards and illustrative reliability scores. No real buyer verification.

7. FPO Aggregation

Previews pooled quantities; member consent and persistent aggregation are not implemented.

8. Logistics

Shows local order-status controls; no live transport quotes, GPS or carrier integration.

9. Transactions & Disputes

Shows sample transactions and accepts local dispute submissions; no real payments or persistent records.

10. Admin & Trust Center

Displays sample verification queue and dispute oversight.

5. System Architecture

```
Farmer / FPO / Buyer / Admin
          |
          v
React + Vite Frontend
(search, lots, net calculator, scenarios, demo workflows)
          |
          | planned REST / JSON connection
          v
Node.js + Express API
(validation, business rules, data endpoints)
          |
          v
MySQL Database
users, FPOs, lots, market prices, forecasts, buyers,
transactions, costs, logistics, disputes, audit logs
          |
          +-- Future official market-data sources
          +-- Future Python ML forecasting service
          +-- Future maps / logistics / payment providers
```

6. Database Design

The schema includes users, FPOs, FPO members, produce lots, quality checks, market prices, price forecasts, buyers, FPO aggregations, aggregation contributions, transactions, cost estimates, logistics, disputes and audit logs. Foreign keys link users, lots, transactions and related records.

```
users 1---* produce_lots *---1 fpos (optional)
produce_lots 1---* transactions *---1 buyers
transactions 1---* logistics
transactions 1---* disputes
fpos 1---* fpo_aggregations 1---* aggregation_contributions
produce_lots 1---* quality_checks
market_prices and price_forecasts store dated market signals
```

7. Net Realization

Gross value = quantity × offer price. Net realization = gross value – transport – commission – storage – other costs. Comparing net proceeds can produce a different ranking from comparing listed prices alone.

8. Forecasting and Sale-Window Recommendation

A future ML service should use source-validated historical price data, define a forecast horizon, validate with time-based train/test splits, report error metrics, and display uncertainty. A sale-window recommendation should compare expected net proceeds after holding costs and clearly state that future prices are not guaranteed. The current Sell-Time Advisor uses a user-entered later price and is scenario arithmetic only.

9. Buyer Trust and FPO Aggregation

Buyer verification requires evidence and a review process. Reliability indicators should be explainable and based on fair signals such as completed transactions, on-time payments, cancellations and substantiated disputes, with an appeal process. FPO aggregation should retain member-level lot quantities, consent, ownership and settlement allocation.

10. Implementation Status

Capability	Status
Navigation and responsive UI	Implemented in browser
Lot search/filter and create lot	Implemented in local React state
Net realization with separate costs	Implemented locally
Sell-time comparison	User-entered scenario; not ML
FPO aggregation	Preview only
Buyer reliability and verification	Illustrative sample values
Mandi prices and history	Simulated; no live feed
Transactions and disputes	Local demo state; not persisted
Express API and MySQL schema	Starter source provided; requires setup
Frontend-to-API integration	Not implemented
Payments, maps, real verification, trained ML	Not implemented

11. Testing Checklist

• Search and filter crop lots.

• Create a lot and verify it appears in the session.

• Change transport, commission, storage and other costs; check net realization.

• Compare sell-now and user-entered later scenarios.

• Preview an FPO aggregation quantity.

• Create a transaction, update logistics status and submit a dispute.

• After server setup, test GET /api/health, /api/lots, /api/buyers and /api/transactions.

12. Limitations, Safety and Security

All market prices, historical charts, buyer reliability scores and verification labels are illustrative. No actual money is collected. No trained ML model or external provider is connected. Browser state resets on refresh and the frontend is not wired to the API. Production requires authentication, role-based authorization, input validation, secure secrets, HTTPS, audit logs, evidence handling, backups and payment-provider integration.

13. Future Roadmap

• Connect a permitted official mandi price source and preserve source/date/unit metadata.

• Train and validate a short-term forecast model and show uncertainty.

• Implement a cost-aware sale-window engine.

• Build real buyer verification and transparent reliability scoring.

• Persist FPO member contributions, lot ownership and consent.

• Connect transport quotes, delivery proof, transaction persistence and dispute workflow.

• Add secure payments, role-based access, audit logging and automated tests.

14. Conclusion

MandiSetu is designed to be more than a listing site: its differentiator is combining price intelligence, net realization, trust and transaction enablement. The current prototype demonstrates the intended workflow and is explicit about simulated features; live data, machine learning and production-grade integrations remain future work.